Ensemble Methods and Random Forests

Albert Y. Kim

Limitations of Decision Trees

- Decision trees are easy to interpret and don't require much computation to train
- However, capturing a complex relationship using a decision tree requires the tree be deep (lots of splits). This is problematic because trees exhibit high variance and are prone to overfitting
- This presentation will focus on the *random forest* algorithm, a well-known *ensemble model*

Ensemble Model

Ensemble modeling combines **multiple** individual base models into a single, comprehensive predictor to improve overall accuracy and reduce error. i.e. the wisdom of crowds.

Each base model contributes towards a final prediction, either by:

- majority/weighted voting (classification)
- simple/weighted averaging (regression).

Types of Ensemble Models

The simplist ensemble model is a voting ensemble (averaging ensemble): Combine/average several different models:

- `VotingRegressor()`: Average the numerical $\widehat{y}$ output of several $\widehat{f}$ models
- `VotingClassifier()`: Combine the output of several $\widehat{f}$ models via either
  - Soft voting: Average (or sum) the predicted probabilities $\widehat{p}$, then pick the class with the largest $\widehat{p}$
  - Hard voting: Simple majority voting of the predicted $\widehat{y}$

In today's lab

You will fit simple ensemble methods:

- Question 1: Work with Wisconsin Breast Cancer data with y binary: malignant (cancerous) or benign:
  - Use classification accuracy as metric
  - Use `VotingClassifier()` to combine the results of the following models via `voting='soft'`: logistic regression, decision trees, KNN with k=5, KNN with k=15
- Question 2: Work with Iowa house prices:
  - Use (negative) RMSE as metric (i.e. bigger is better)
  - Use `VotingRegressor()` to average the predictions of the following models: linear regression, decision trees, KNN

In both cases you'll be using `GridSearchCV()` to do a grid search over a combination of parameters

Types of Ensemble Models

Other types include:

- **Stacking** (Stacked Generalization): Not covered in this class.
- **Boosting**: Trains models sequentially, where each new model focuses on correcting the errors or misclassified samples made by the previous one. Ex: XGBoost (later)
- **Bagging** (Bootstrap Aggregating): Trains multiple homogeneous models in parallel on bootstrap resamples of the data, then averages or votes on their outputs. Ex: Random Forests (today)

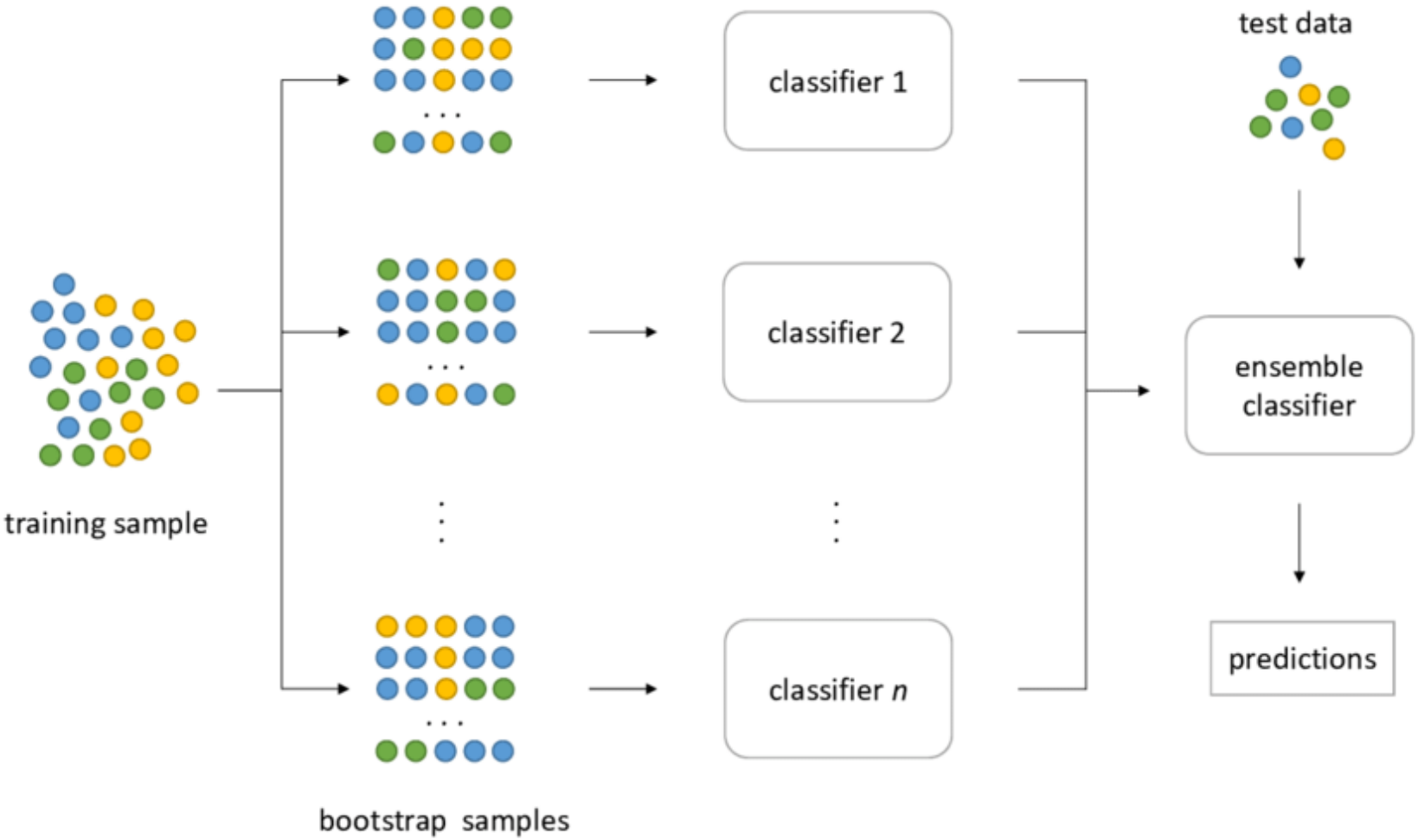
Bagging Ensemble Models

Bagging models, or "bootstrap aggregation", are a general class of models that work in two steps:

1. **Bootstrap:** draw many random samples from the training data *with replacement*, each the same size as the original, so each is a slight variation of it
2. **Aggregate:** train one model on each sample, then combine their predictions by averaging (regression) or voting (classification)

Bagging Ensemble Models

Imagine we have y categorical with levels/classes: blue, green, yellow. (image source)



Random Forests

Random forests are a specific kind of bagging model where:

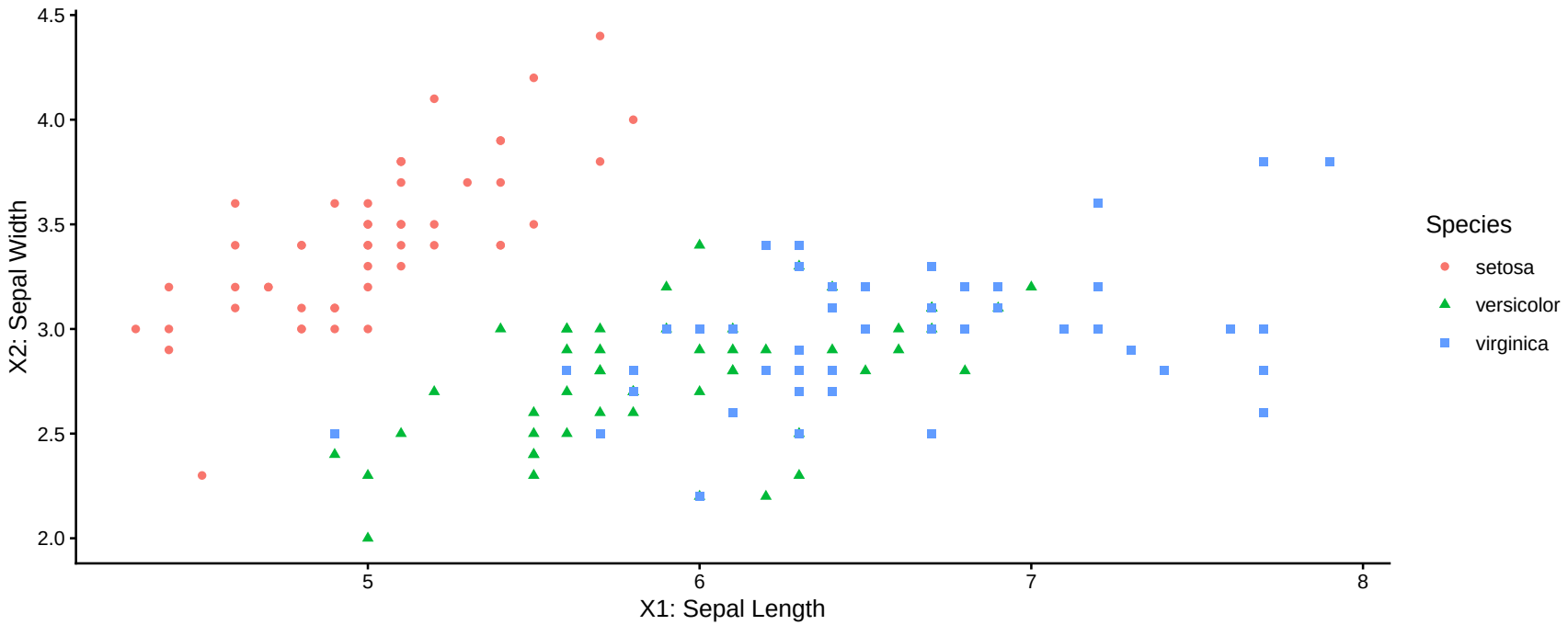
1. Each base model is a decision tree
2. Each base model is fitted to a bootstrap resampled training data
3. Additionally, we have *predictor sampling*: At every split in each tree, instead of considering all features in determining where to split, we randomly sample `max_features` of them.

Class Discussion:

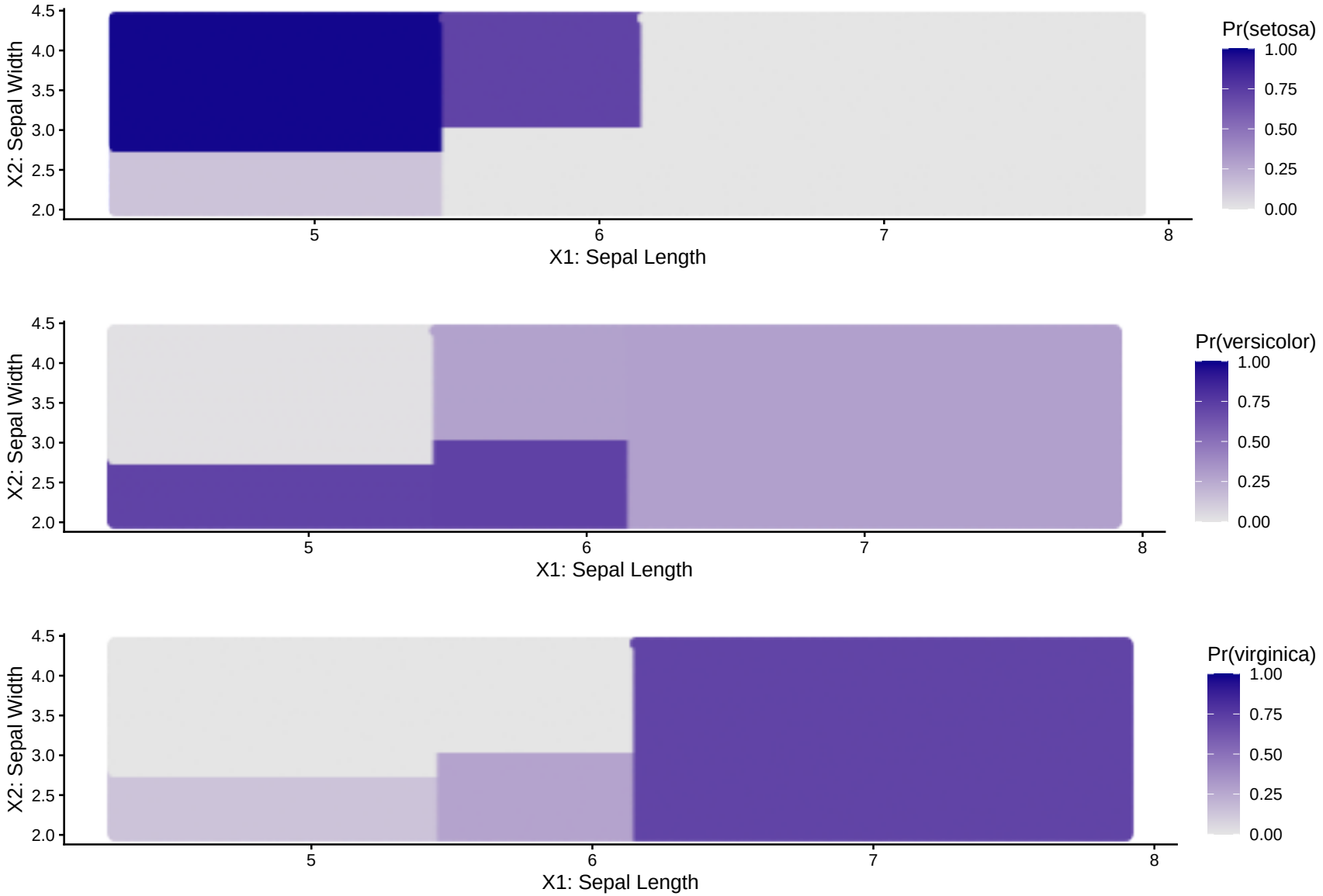
- What would happen if bootstrap resampling with replacement were not used?
- What might happen if predictor sampling were not used?

Previous Data

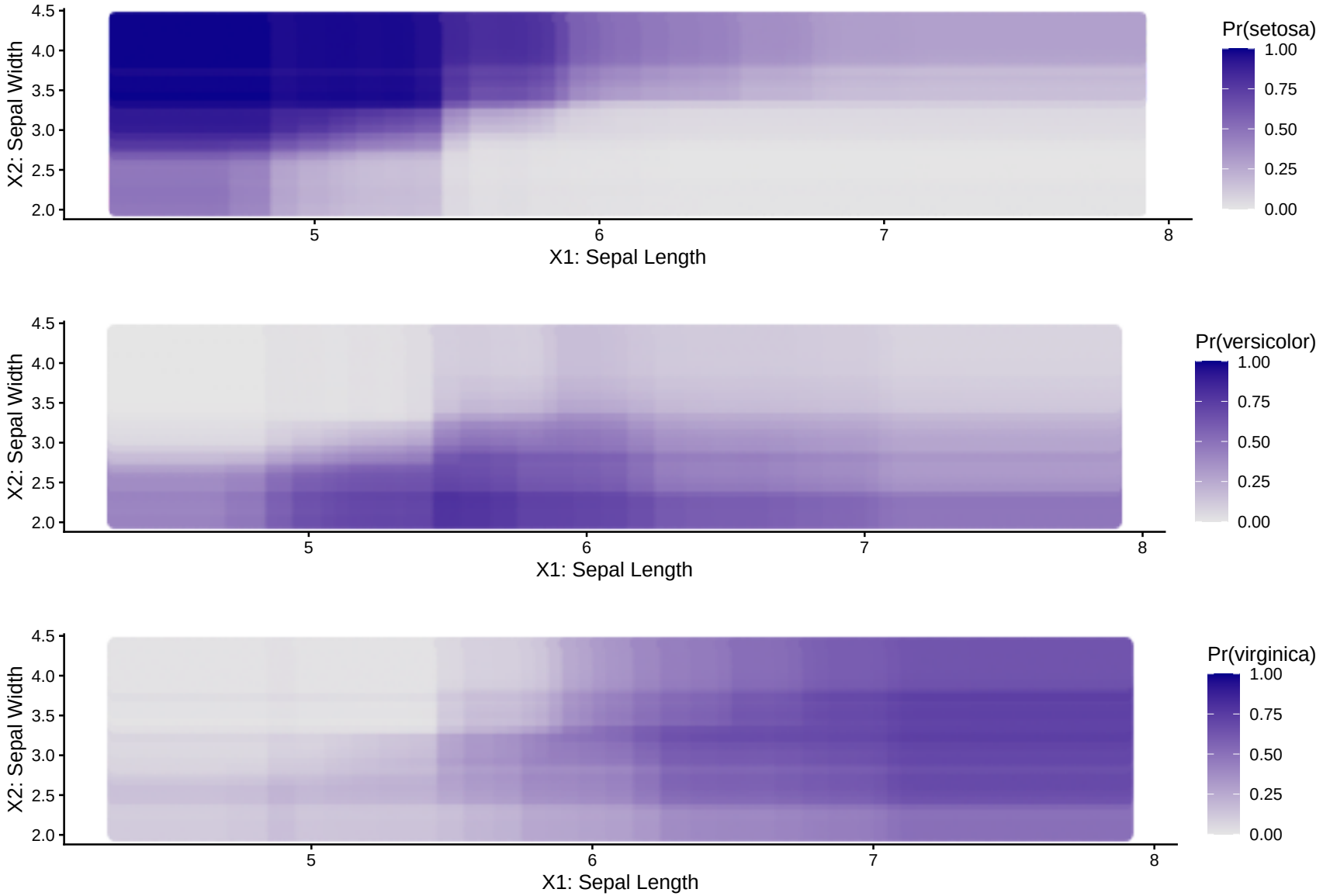
Recall the iris data:



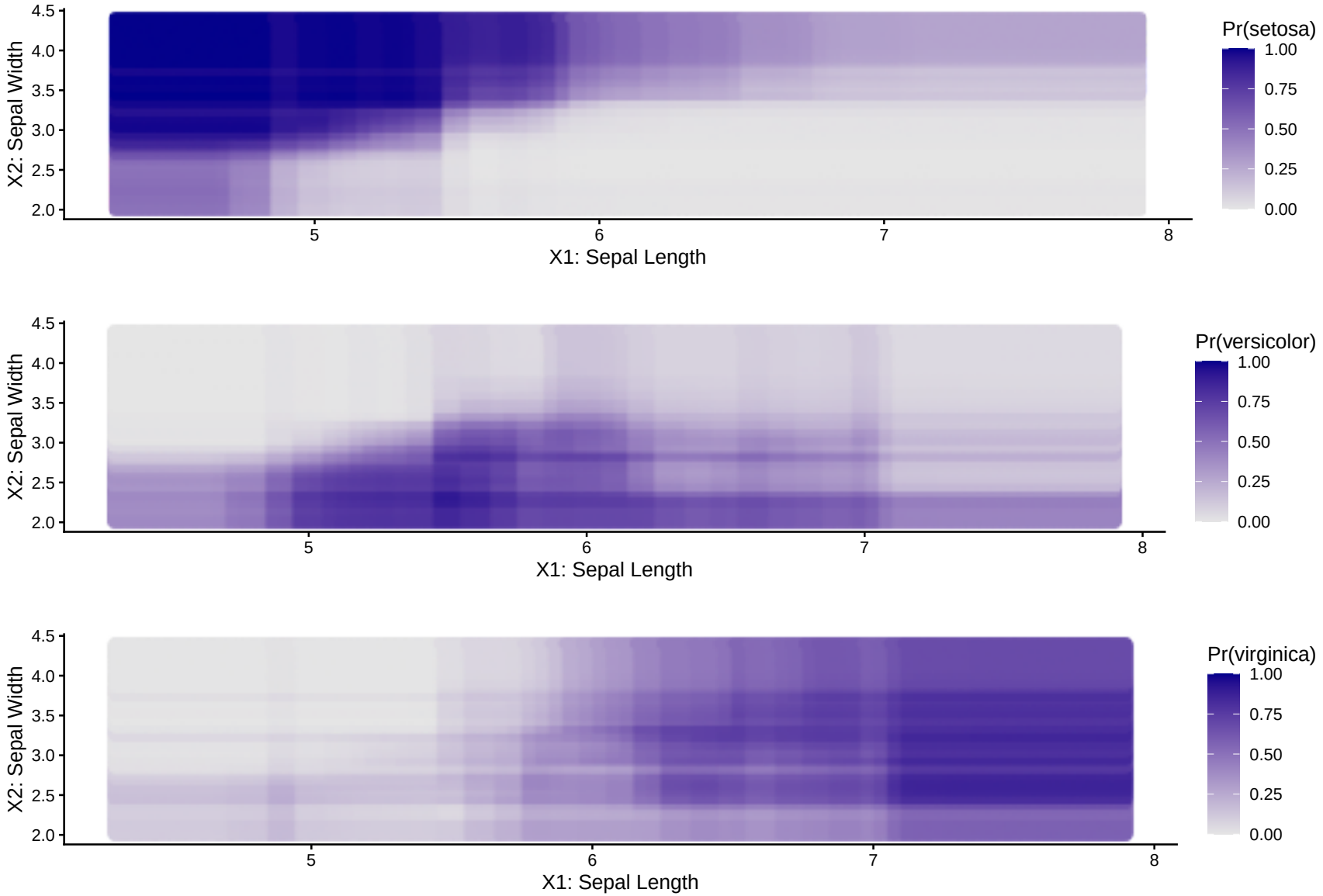
Last Lecture: Single Decision Tree with max depth



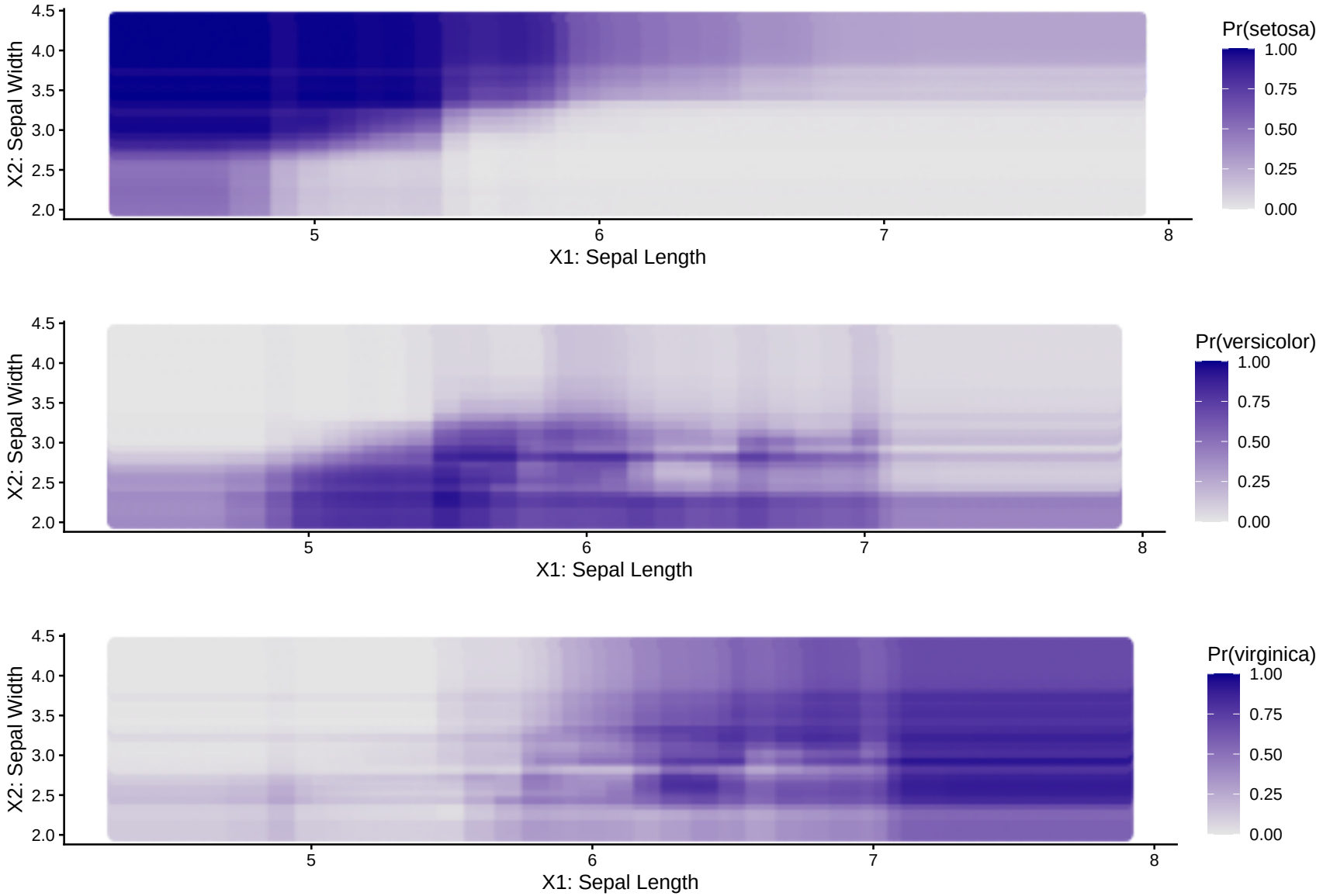
Random Forest with 500 trees & `max_depth` = 3



Random Forest with 500 trees & `max_depth` = 5



Random Forest with 500 trees & max depth = 50



Comparing results

Model	Accuracy on training data
Single tree (max depth = 3)	79.3%
RF: 500 trees, max depth = 3	82.0%
RF: 500 trees, max depth = 5	84.0%
RF: 500 trees, max depth = 50	84.7%

In today's lab

You will grow your first random forest:

- Question 3: Work with the Iowa house prices again
  - Use (negative) RMSE as the metric again (i.e. bigger is better)
  - Use `GridSearchCV()` to tune a `RandomForestRegressor()` that predicts house prices $\widehat{y}$
  - Compare its `cross_val_score()` (negative) RMSE with your best ensemble from Question 2

Final Remarks about Random Forests

- Pros:
  - Generally offer better predictive performance than a single decision tree
- Cons:
  - Are not easily interpretable
  - Still prone to overfitting
- Tuning parameters
  - `max_depth` and `min_samples_split` just like with regular decision trees
  - `max_features`: the number or fraction of predictors considered at each split
- Also you can set the number of trees `n_estimators`, but including more trees past a certain point will not improve the ensemble

Final Remarks (cont.)

- `max_depth` and `min_samples_split` help prevent base models from being overfit
  - By using an ensemble approach, random forests can be flexible without using deep trees
  - Thus, relative to a single decision tree, you should consider using a smaller `max_depth` and larger `min_samples_split`
- `max_features` governs the degree of correlation between base models
  - Smaller values reduce correlations between trees (at the expense of predictive power within individual trees)
  - Default recommendations are $\sqrt{p}$ for classification and $p/3$ for regression